

DOCUMENTEACIÓN
TRABAJO — CAFETERÍA MULTI-HILOS

Eder Martínez Castro

31 de octubre de 2025

Índice general

1.	Análisis y Especificación de Requisitos	2
1.1.	Descripción del problema	2
1.2.	Requisitos funcionales	2
1.3.	Requisitos no funcionales	2
1.4.	Casos de uso	3
1.5.	Historias de usuario	3
2.	Arquitectura del sistema	4
2.1.	Arquitectura	4
2.2.	Clases	5
2.3.	Stack empleado	6
2.4.	Interfaz	6

1. Análisis y Especificación de Requisitos

1.1. Descripción del problema

Se requiere un **programa en Java** que **simule una cafetería** utilizando **hilos**, los **clientes** llegaran a la cafetería y serán atendidos por los **camareros**.

- Cada **cliente** tendrá nombre y **tiempoEspera**.
- Los **camareros** prepararán los cafés en orden de llegada.
- Si el tiempo de espera del cliente se acaba, **se marchará**.
- El sistema terminará cuando **todos los clientes han recibido su cafe o se fueron**.

Alcance: Se implementarán 2 versiones:

- **Terminal:** Simulación simple por terminal.
- **Interfaz:** Se creará una interfaz simple en **JavaFX**

1.2. Requisitos funcionales

A continuación, se listan los requisitos funcionales del sistema:

ID	Descripción
RF-01	Crear clientes: nombre:String y tiempoEspera:int (ms).
RF-02	Crear camareros: instanciar n camareros, cada uno se ejecutará en un hilo independiente.
RF-03	Preparar café: el camarero simulara con <code>Threads.sleep</code> el tiempo que tarda en prepararlo.
RF-04	Abandono por espera: si el cliente no recibe su café y se le acabo su tiempoEspera , se marchará.
RF-05	Entrega del café: cuando el camarero haya terminado de hacer el café y el cliente siga esperando se le entregará.
RF-06	Fin del programa: cuando todos los clientes hayan recibido su cafe o se hayan ido, el programa terminará.

1.3. Requisitos no funcionales

A continuación, se listan los requisitos no funcionales del sistema:

ID	Descripción
RNF-01	Lenguaje: El programa se realizará en Java y JavaFX .
RNF-02	Observabilidad: Los mensjes mostrados deben ser claros y simples.
RNF-03	Cierre ordenado: todos los hilos terminaran de forma correcta antes de que termine el programa.

1.4. Casos de uso

Descripción de casos de uso principales:

Caso de uso	Descripción
CU-01 Llegada y pedido	El cliente llega, y lo atiende el camarero y esperar su café hasta que se lo de o su tiempoEspera acabe.
CU-02 Preparación	Un camarero se le asigna un cliente de la lista y ejecuta prepararCafe(cliente) simulando el tiempo que le lleva hacer el café.
CU-03 Entrega	Si el cliente no se le termino el tiempo de espera, el camarero le entrega el café y se muestra el tiempo total de preparación de ese café por parte del camarero.
CU-04 Abandono	Si el tiempo de espera termina antes de la entrega, el cliente se marchará y el camarero pasa al siguiente cliente.
CU-05 Fin del programa	Cuando ya todos los clientes recibieron su café o se fueron y los camareros ya no están preparando cafés.

1.5. Historias de usuario

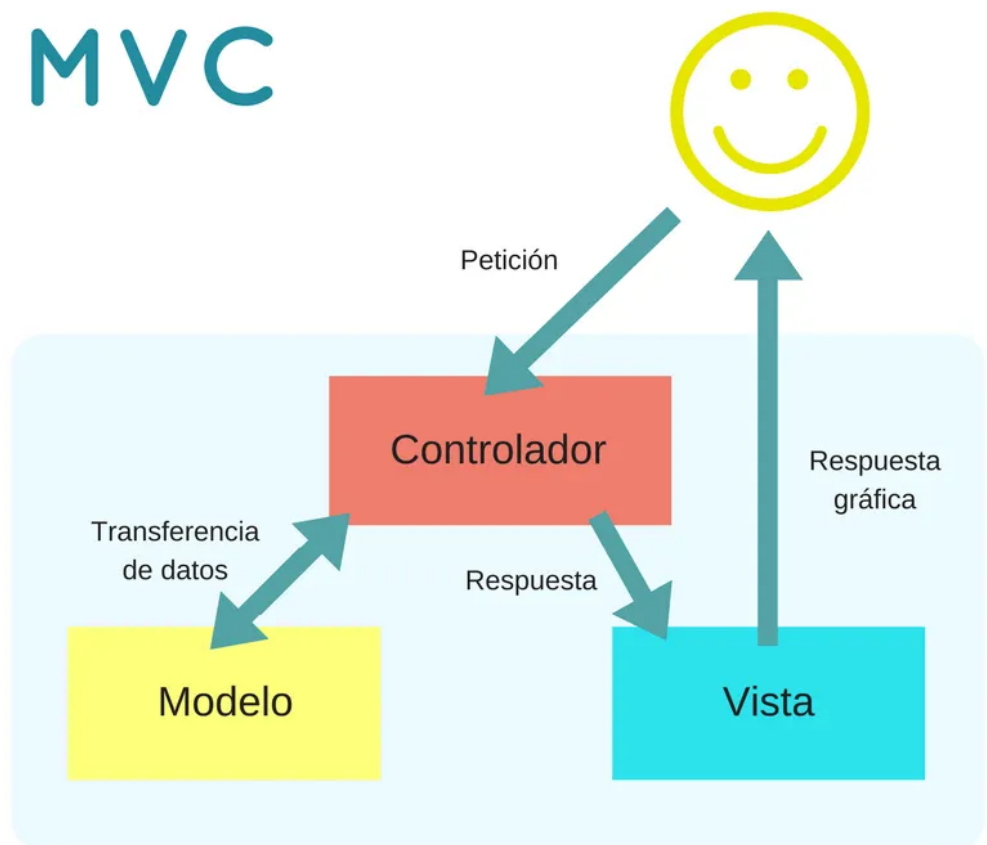
ID	Como...	Quiero...	Para...
HU-01	Cliente	Pedir mi café y esperar un tiempo límite	No perder el tiempo si no me atienden me marchó.
HU-02	Camarero	Atender a los cliente uno por uno	Empezar a servir por orden de llegada.
HU-03	Usuario	Poder ver el estado del programa	Saber que clientes están atendidos y por quien y saber quien recibe su café o se marcha.

2. Arquitectura del sistema

2.1. Arquitectura

La arquitectura de esta aplicación se explicará en base a lo realizado en JavaFX ya que sigue una arquitectura **Modelo–Vista–Controlador (MVC)** sencillo que corre únicamente en local.

- **Modelo:** cuenta con las clases `cliente` y `camarero` con la lógica, los camareros preparan los cafés cuando tienen un cliente al que están atendiendo y notifican cuando terminan y el cliente espera su café hasta que se lo sirvan o se agote su tiempo de espera y se marche.
- **Vista:** muestra los `clientes` y `camareros` con sus estados si están esperando, atendidos, si les sirvieron o si se fueron y si están libres o atendiendo un cliente.
- **Controlador:** Responde al botón de iniciar la simulación y actualiza los `contenedors` para mostrar el estado de los clientes y camareros en la interfaz.



 coding or not

Figura 1: Arquitectura MVC

2.2. Clases

En este caso tenemos 2 clases principales la de **Clientes** y la de **Camareros**:

Cliente/Client

Client representa a un cliente que entra en la cafetería, pide su café y espera un tiempo máximo (**waitingTime**) antes de marcharse si no lo recibe.

Atributos

- **Identificación:** **name** (nombre del cliente), **waitingTime** (tiempo que está dispuesto a esperar en ms).
- **Estados:** **inFile** (si está en la cola), **assigned** (si está atendido), **served** (si ya le sirvieron el café), **left** (si ya se marchó).
- **Tiempos:** **arriveAt** (instante en el que llega), **timeStart** (inicio del tiempo de espera), **preparationTime** (cuanto tardo el camarero en hacer su café).

Flujo: Al arrancar, marca la llegada del cliente (**timeStart**, **arriveAt**, **inFile** = **true**) y entra en un bucle esperando a que **served** pase a **true**. al mismo tiempo, comprueba si el tiempo de espera actual supera al **waitingTime**, si ocurre, marcamos **left** = **true** y termina. Si el camarero le entregó el café a tiempo, muestra un mensaje de éxito con el tiempo de preparación (**preparationTime**) que el camarero escribió para este cliente.

Camarero/Waiter

Waiter representa a un camarero que atiende a un solo cliente asignado cada vez.

Atributos

- **Identificación:** **name** (nombre del camarero).
- **Control:** **assignment** (cliente asignado actual), **running** (permite poder parar el bucle), **preparationTime** (tiempo que tarda el camarero en hacer el café).
- **Temporización:** **timeStart** (para medir la duración desde que empieza con la preparación).
- **Simulación:** generador aleatorio (**Random**) para la preparación ya que no se tarda lo mismo siempre en hacer un café.

Flujo: El camarero trabaja mientras **running** sea **true** o quede un cliente asignado. Si no hay cliente (**assignment** == **null**), está inactivo unos milisegundos y vuelve a comprobar para ver si puede atender. Cuando hay un **Cliente**, primero valida si el cliente ya **left** (si ya se marchó) antes de empezar. si es así, pasá al siguiente cliente. Si todavía sigue, empieza a atender, y mide el tiempo de preparación (**prepareCoffe()**) y, al terminar, si el cliente no se marchó, guarda **preparationTime** y marca **client.served** = **true**. Pero si el cliente se marchó durante la preparación, lo mostramos por la terminal. Y finalmente, deja **assignment** = **null** para poder atender a otro cliente.

2.3. Stack empleado

Para el desarrollo nuestra aplicación tanto en terminal como con interfaz, se utilizaron las siguientes tecnologías:

- Utilizamos Java como **entorno de ejecución** principal.
- Utilizamos JavaFX para crear una **interfaz con Java**.

2.4. Interfaz

Se creo una interfaz muy simple en JavaFX, para **mostrar el programa de forma clara y bonita**, para saber cuando son atendidos los clientes y cuanto tardan en recibir su café o si se han ido. Como podemos apreciar tenemos un **título** y debajo el **botón** para comenzar con esta simulación del servicio de la cafetería y luego tenemos **dos contenedores** uno para clientes y el otro para los camareros donde mostramos su estado.



Figura 2: Interfaz en JavaFX